**Year: 2026-2027** **Sem V-NEP**

**Experiment No: 08**

**Date of Performance: 06/10/2026** **Date of Submission: 06/10/2026**
**Class: T1**
**Student Roll No: 2401034** **Student Name: Ishita Gurbaxani**

## Aim:

To build an automated CI/CD pipeline on AWS that fetches source code from GitHub, builds it with AWS CodeBuild, and deploys it to an Amazon S3 static website using AWS CodePipeline.

This experiment maps to LO3.

**LO3:** Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

## Theory:

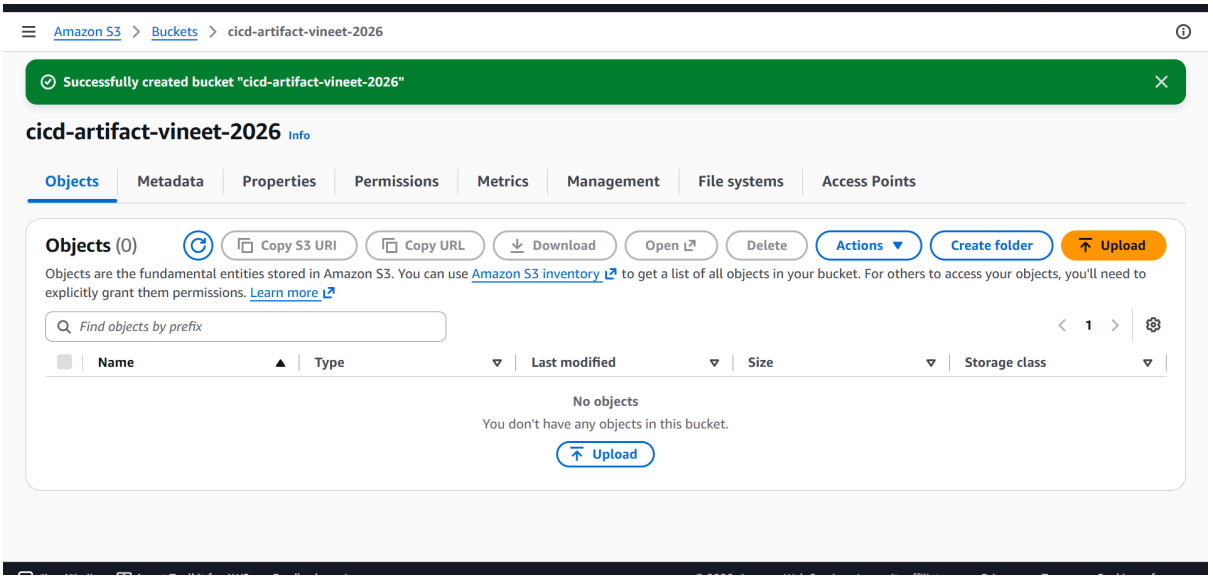
Continuous Integration and Continuous Delivery (CI/CD) is a DevOps practice in which every change committed to a source repository is automatically built, tested, and released through a defined sequence of stages. Continuous Integration ensures that each commit is merged and verified by an automated build, so integration problems surface early, while Continuous Delivery and Deployment carry the verified build through to a target environment without manual intervention. This shortens the feedback loop between writing code and seeing it run, and makes releases small, frequent, and repeatable rather than large and error-prone.

AWS provides a fully managed set of services for implementing such pipelines. AWS CodePipeline is the orchestration service: it models a release process as a series of stages — Source, Build, Test, and Deploy — and passes the output of each stage to the next as an artifact stored in an Amazon S3 artifact bucket. AWS CodeBuild is a serverless build service that provisions a temporary container, pulls the source, and executes the commands declared in a buildspec.yml file at the root of the repository. Amazon S3 serves two roles in this experiment: it stores the intermediate pipeline artifacts, and, with static website hosting enabled and a public-read bucket policy attached, it acts as the deployment target that serves the website over HTTP. Because the pipeline is connected to GitHub through a webhook, every push to the main branch triggers a new execution, so the live website always reflects the latest committed code.

## Implementation:

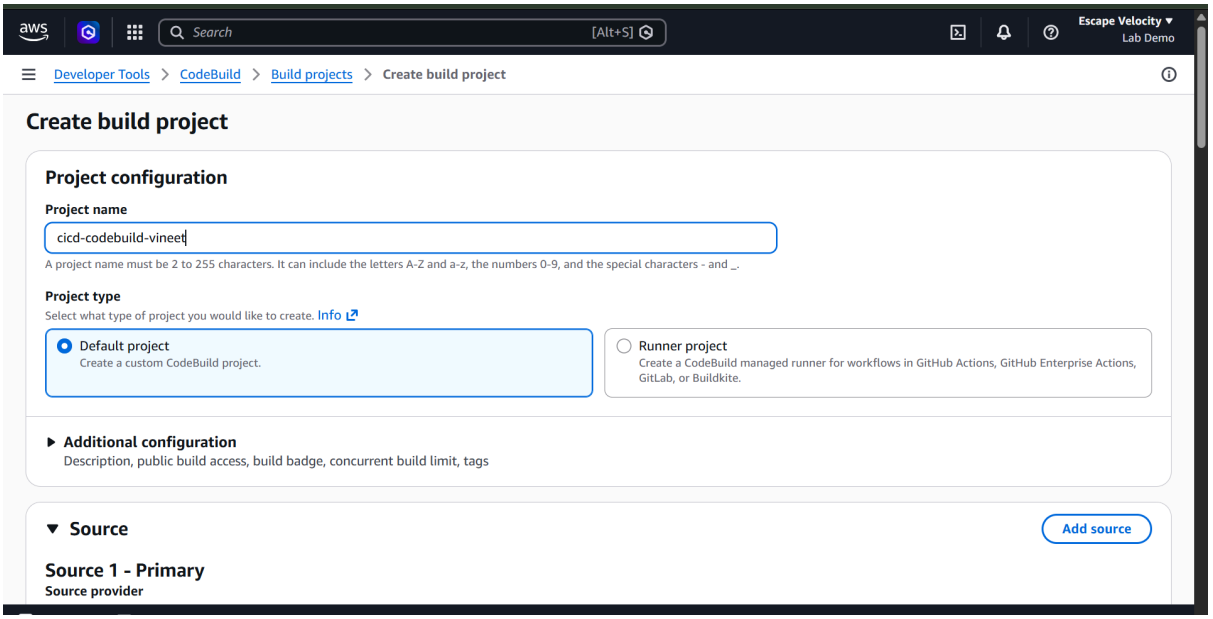
1. Create the Artifact Bucket:

An S3 bucket named cicd-artifact-vineet-2026 is created to act as the artifact store for the pipeline. CodePipeline uses this bucket to hold the source and build artifacts that are passed between stages, so it is created empty before the pipeline itself is configured.



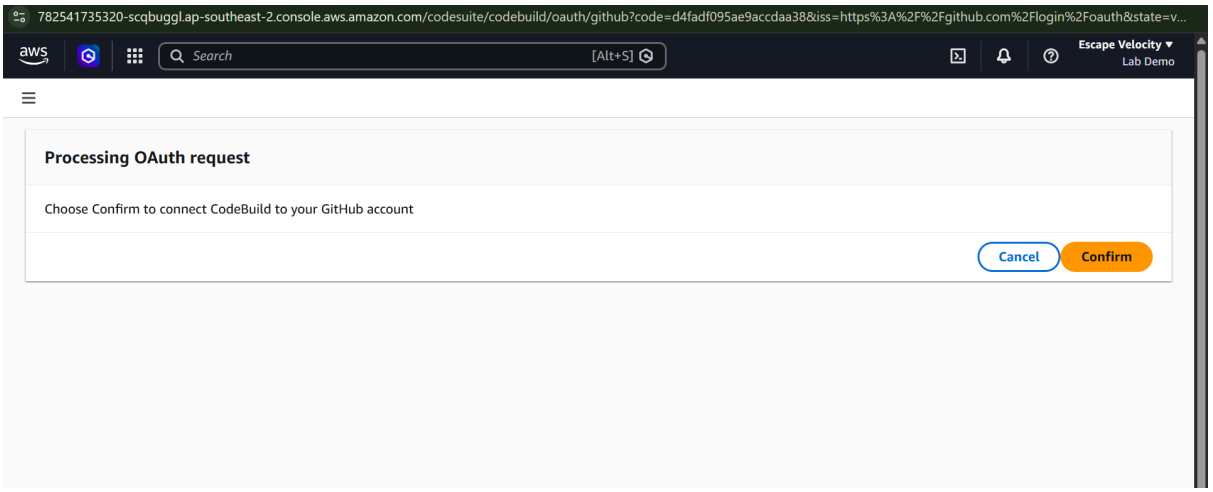

2. Create the CodeBuild Project:

In the CodeBuild console a new build project named cicd-codebuild-vineet is created with the Default project type. This project defines the build environment and the steps that CodeBuild executes whenever the pipeline reaches the Build stage.



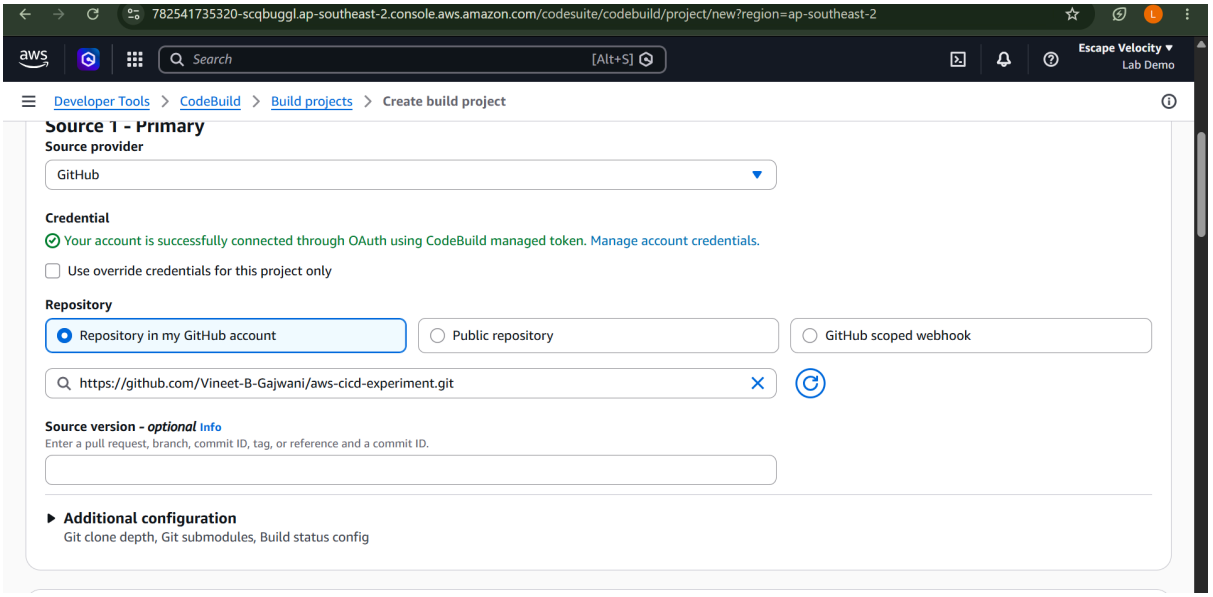
3. Connect CodeBuild to GitHub:

GitHub is selected as the source provider and CodeBuild is authorised through an OAuth request. Choosing Confirm links the GitHub account to CodeBuild using a CodeBuild-managed token, allowing it to read the repositories owned by the account.




4. Select the Source Repository:

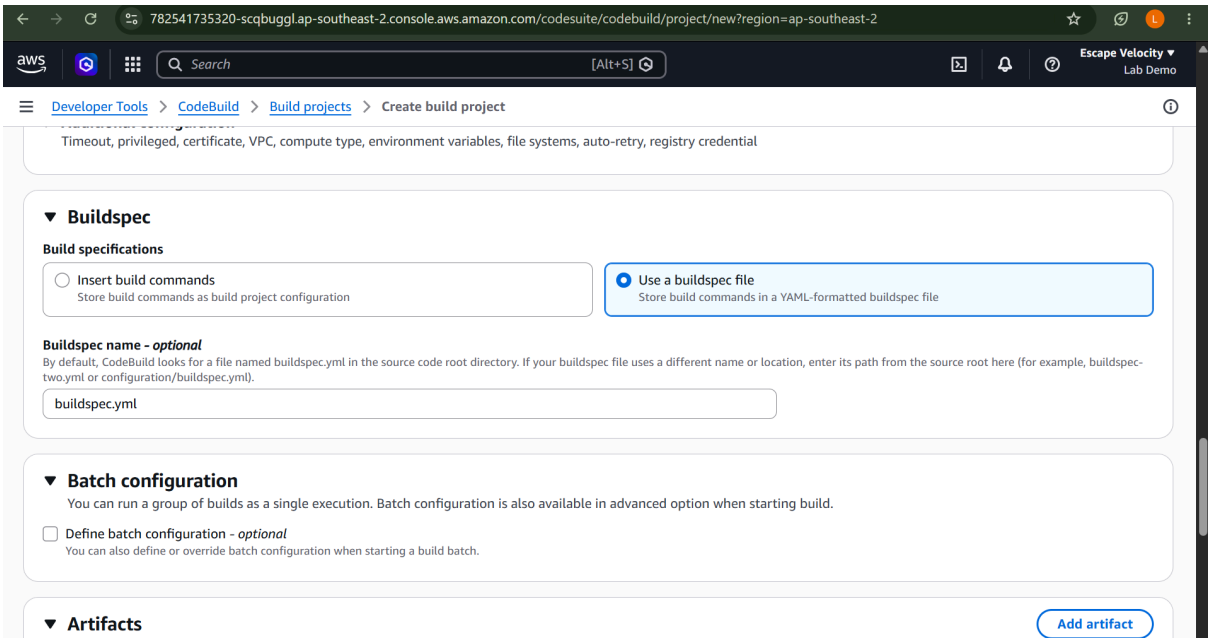
With the OAuth connection established, the option Repository in my GitHub account is chosen and the repository aws-cicd-experiment is selected as the primary source. The source version is left blank so that the default branch is built.




5. Configure the Buildspec:

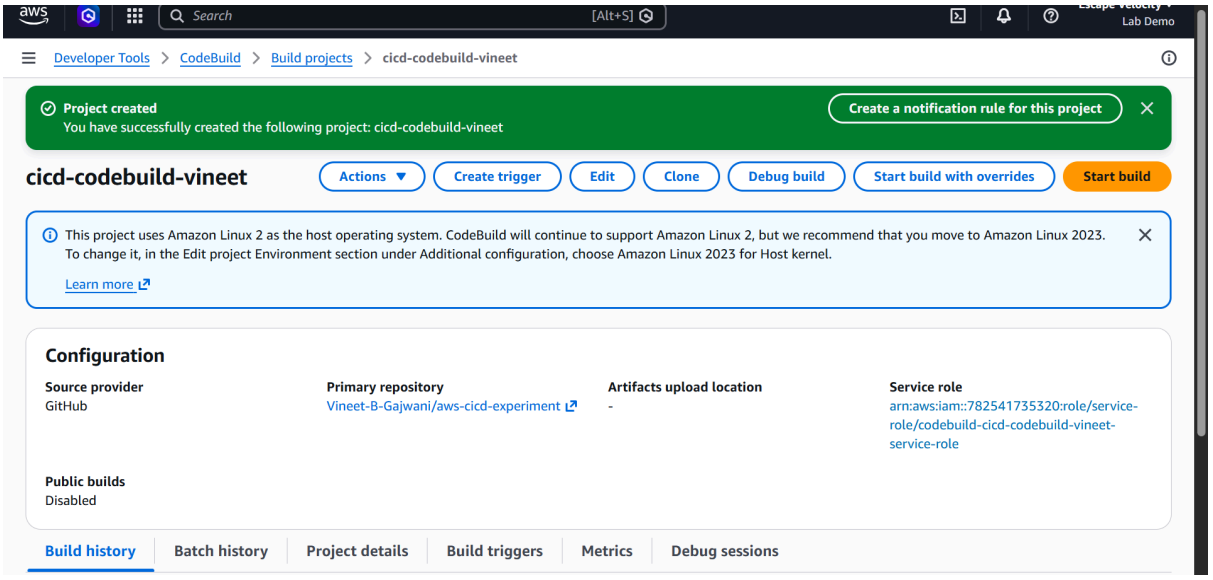
Under Build specifications the option Use a buildspec file is selected, with the file name buildspec.yml. CodeBuild therefore reads its install, build, and artifact instructions from the

YAML file committed in the root of the repository instead of storing commands in the project configuration.



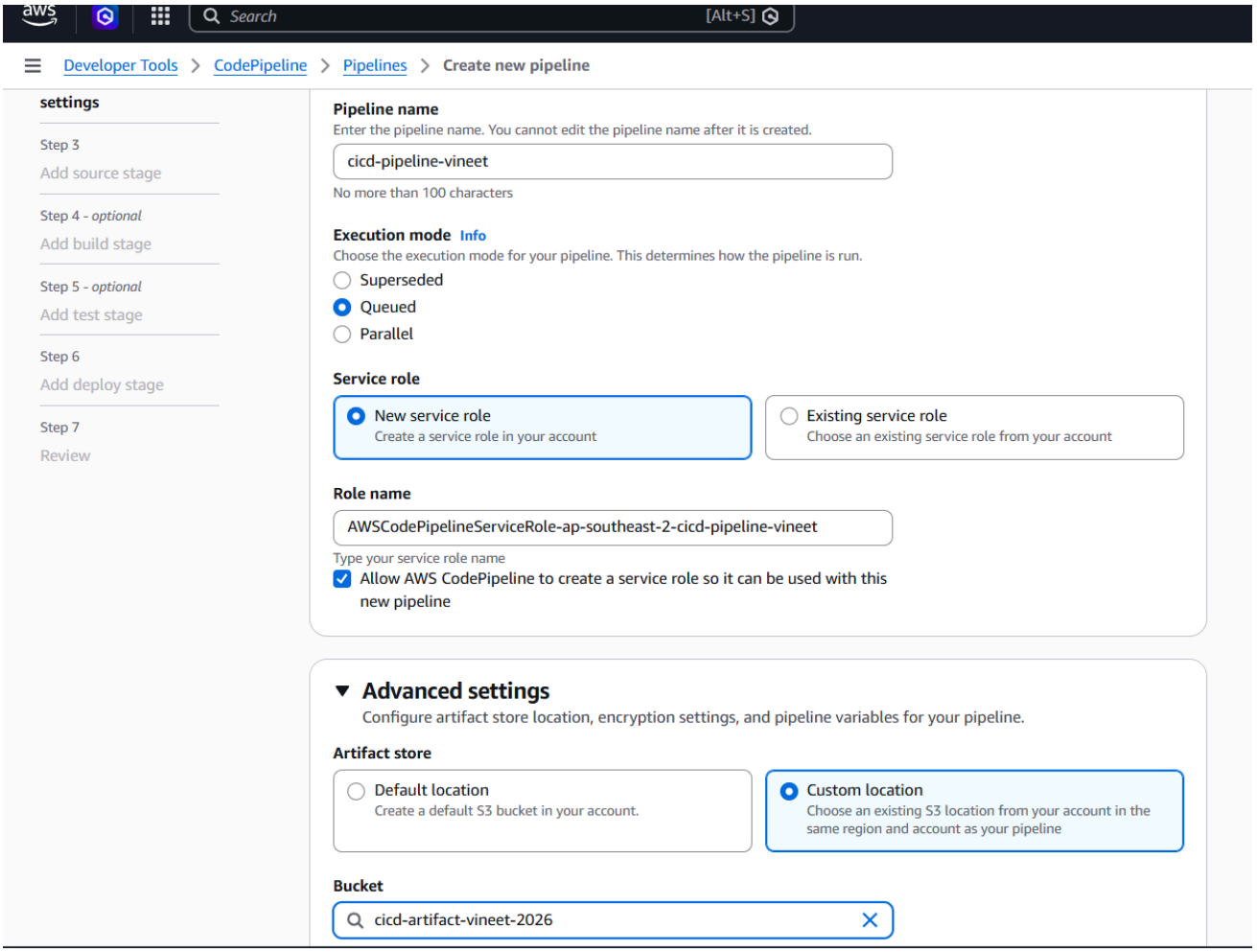

6. Verify the Build Project:

The project is created successfully. The configuration panel confirms GitHub as the source provider, Vineet-B-Gajwani/aws-cicd-experiment as the primary repository, and an automatically generated IAM service role that grants CodeBuild the permissions it needs during a build.



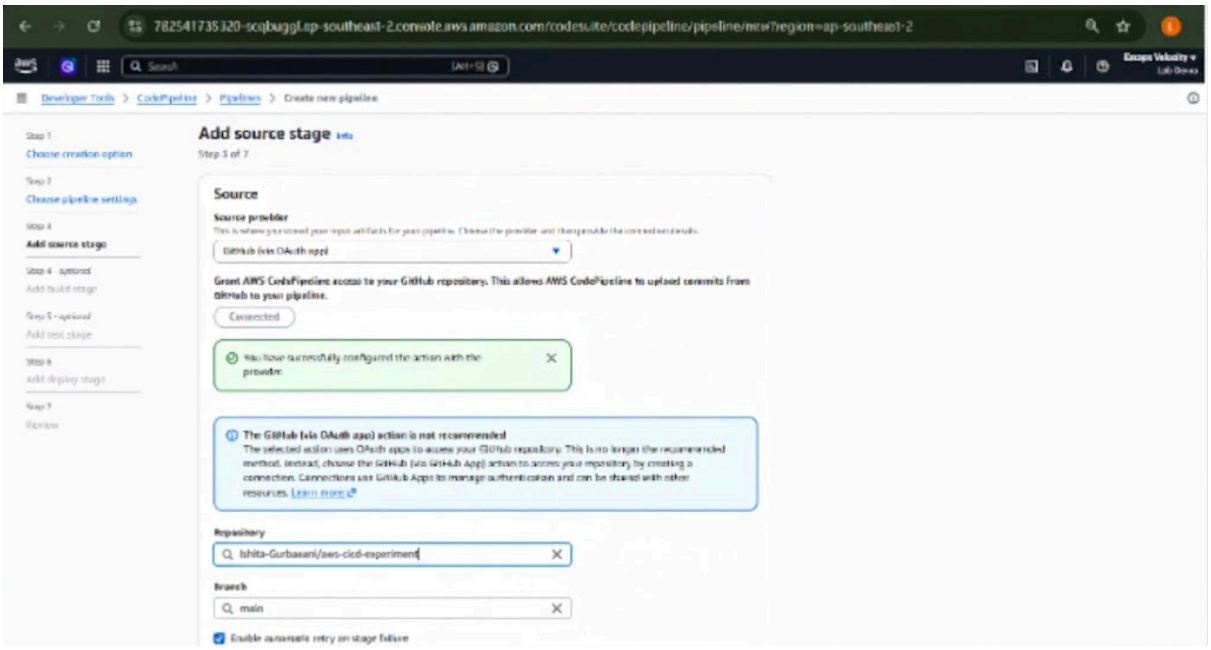
7. Configure the Pipeline Settings:

In CodePipeline a new pipeline named cicd-pipeline-vineet is created in Queued execution mode, together with a new service role. Under Advanced settings a custom artifact store is chosen and pointed at the cicd-artifact-vineet-2026 bucket created in the first step.



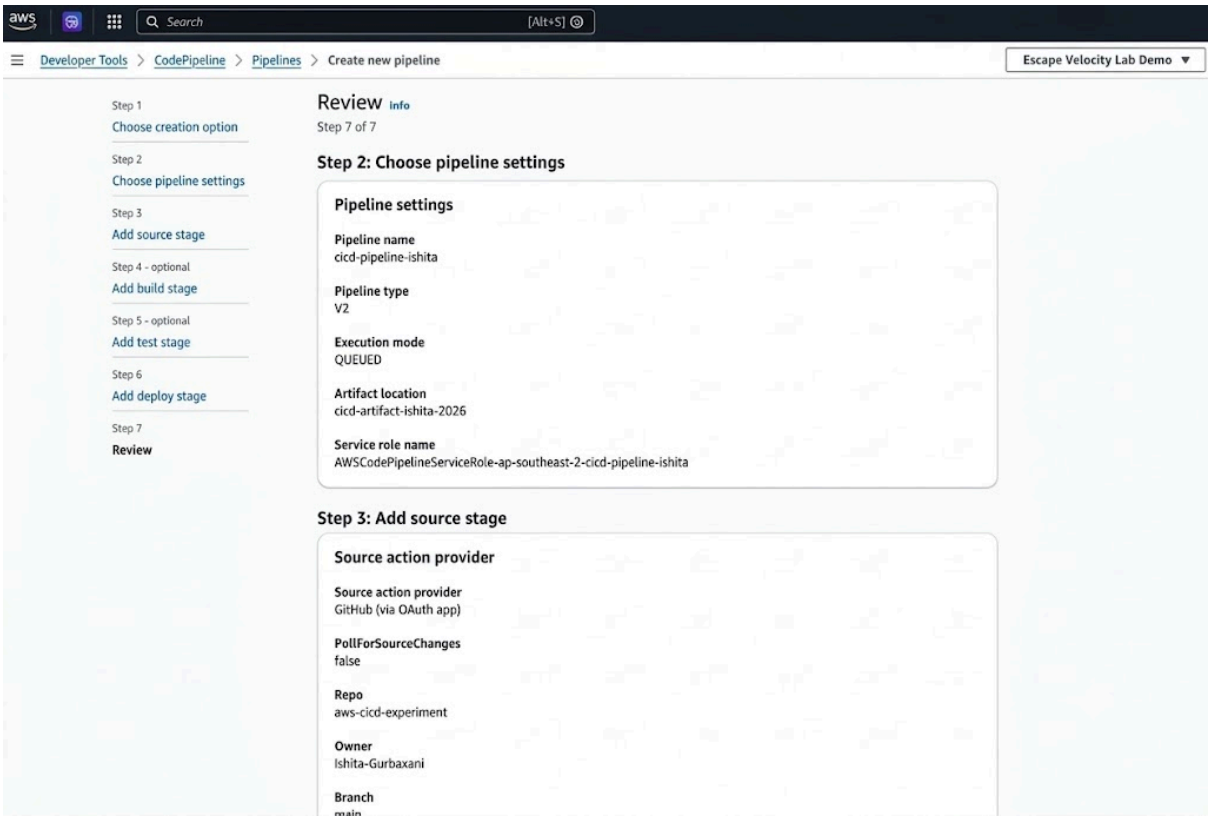
8. Add the Source Stage:

GitHub (via OAuth app) is chosen as the source provider and the connection is confirmed. The repository Vineet-B-Gajwani/aws-cicd-experiment and the branch main are selected, so that every commit to main becomes a new input to the pipeline.



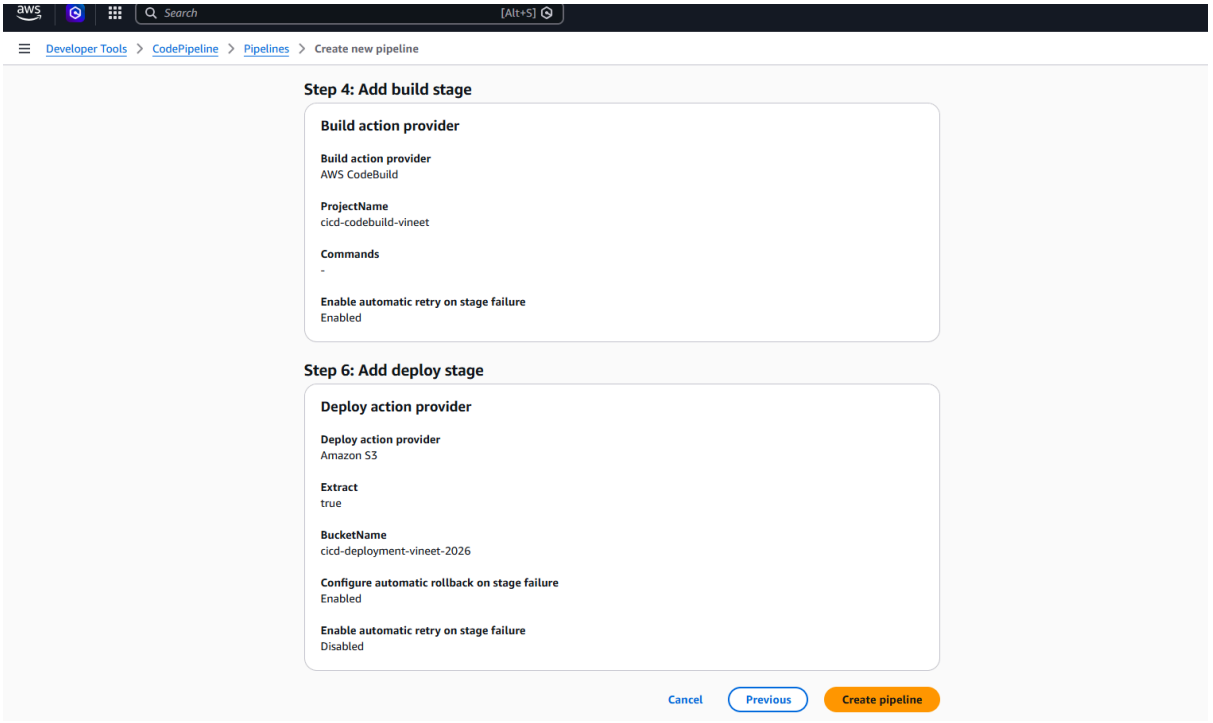
9. Review the Pipeline and Source Settings:

The review page summarises the pipeline configuration: pipeline type V2, execution mode QUEUED, the custom artifact location, and the generated service role. The source stage lists GitHub as the provider along with the repository owner, name, and branch.



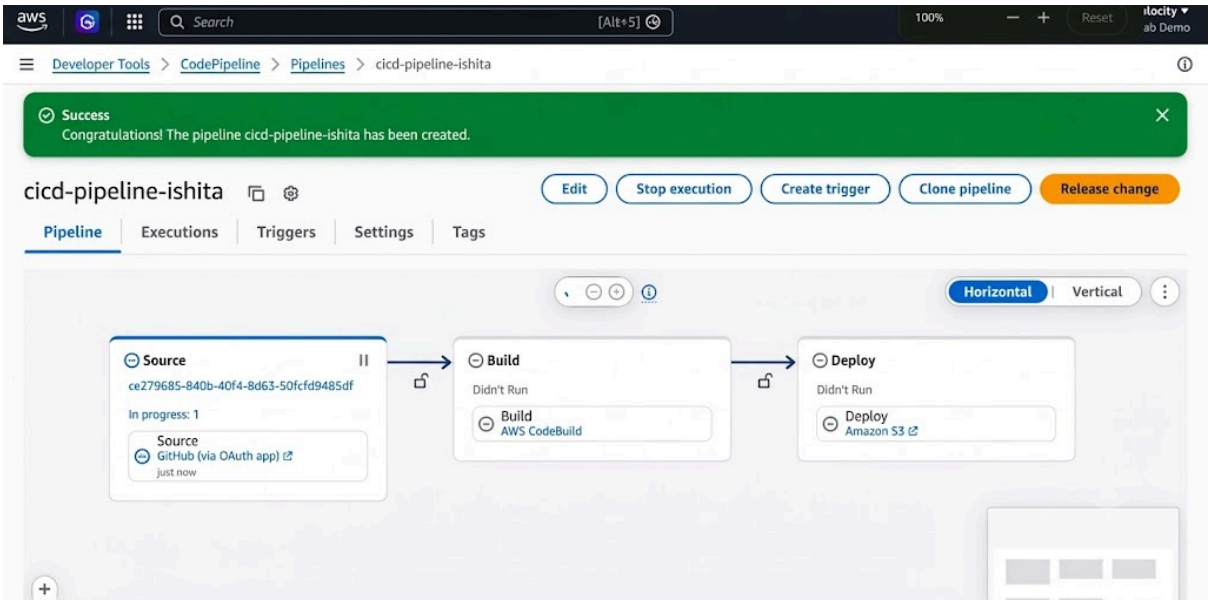
10. Review the Build and Deploy Stages:

The build stage uses AWS CodeBuild with the project cicd-codebuild-vineet, and the deploy stage uses Amazon S3 with the bucket cicd-deployment-vineet-2026. Extract is set to true so that the build output is unzipped into the bucket as individual website files. Choosing Create pipeline saves the configuration.



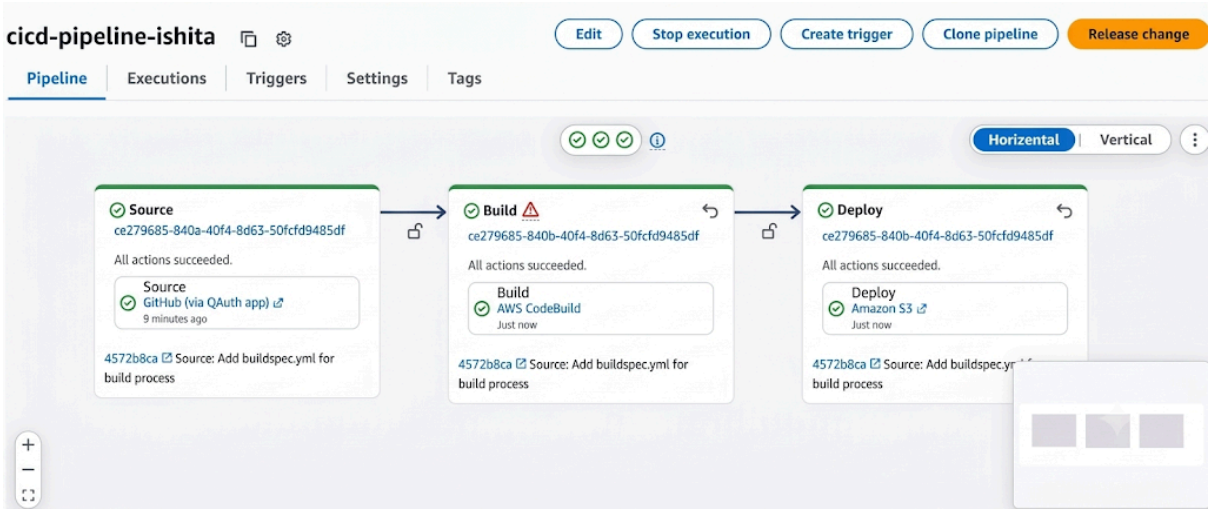
11. Run the Pipeline for the First Time:

The pipeline is created and its first execution starts immediately. The Source stage is in progress fetching the latest commit, while the Build and Deploy stages are waiting for their input artifacts.



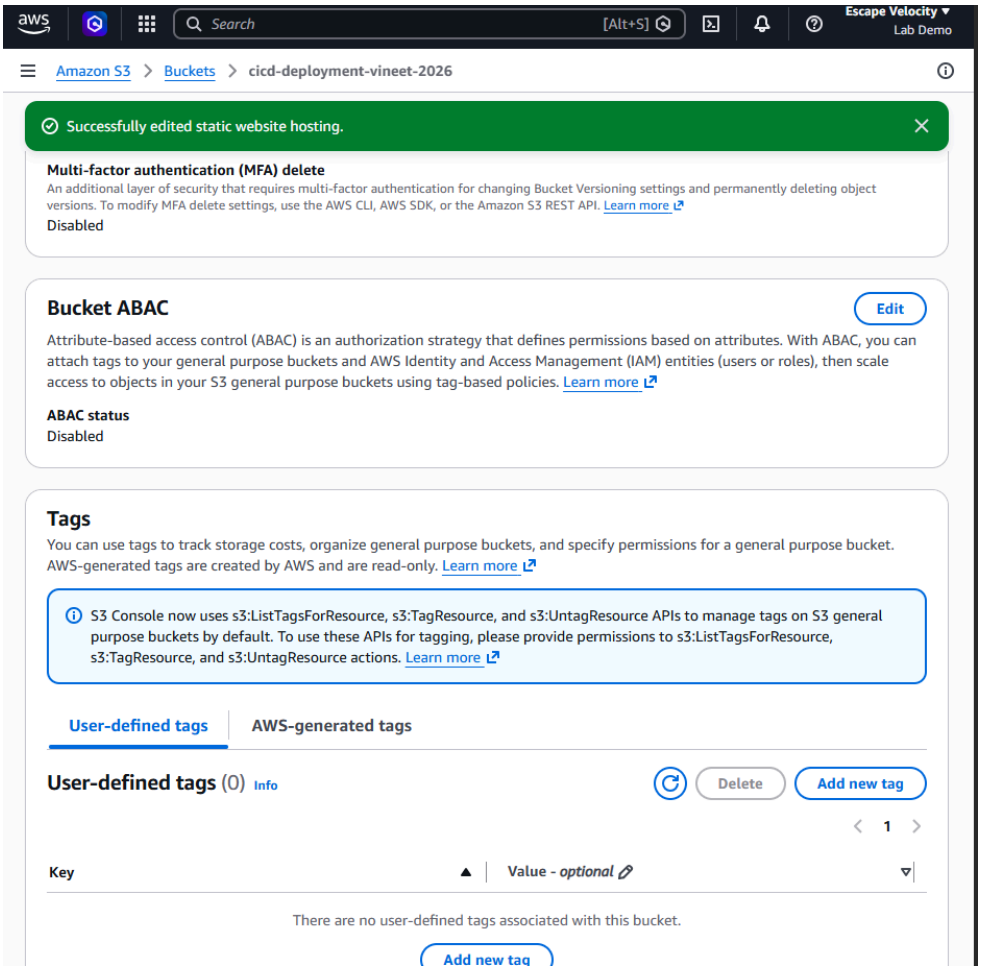

12. Verify a Successful Execution:

All three stages — Source, Build, and Deploy — complete successfully for the commit 4572b8ca, “Add buildspec.yml for build process”. The same commit identifier appears in every stage, showing that a single revision flowed through the entire pipeline.



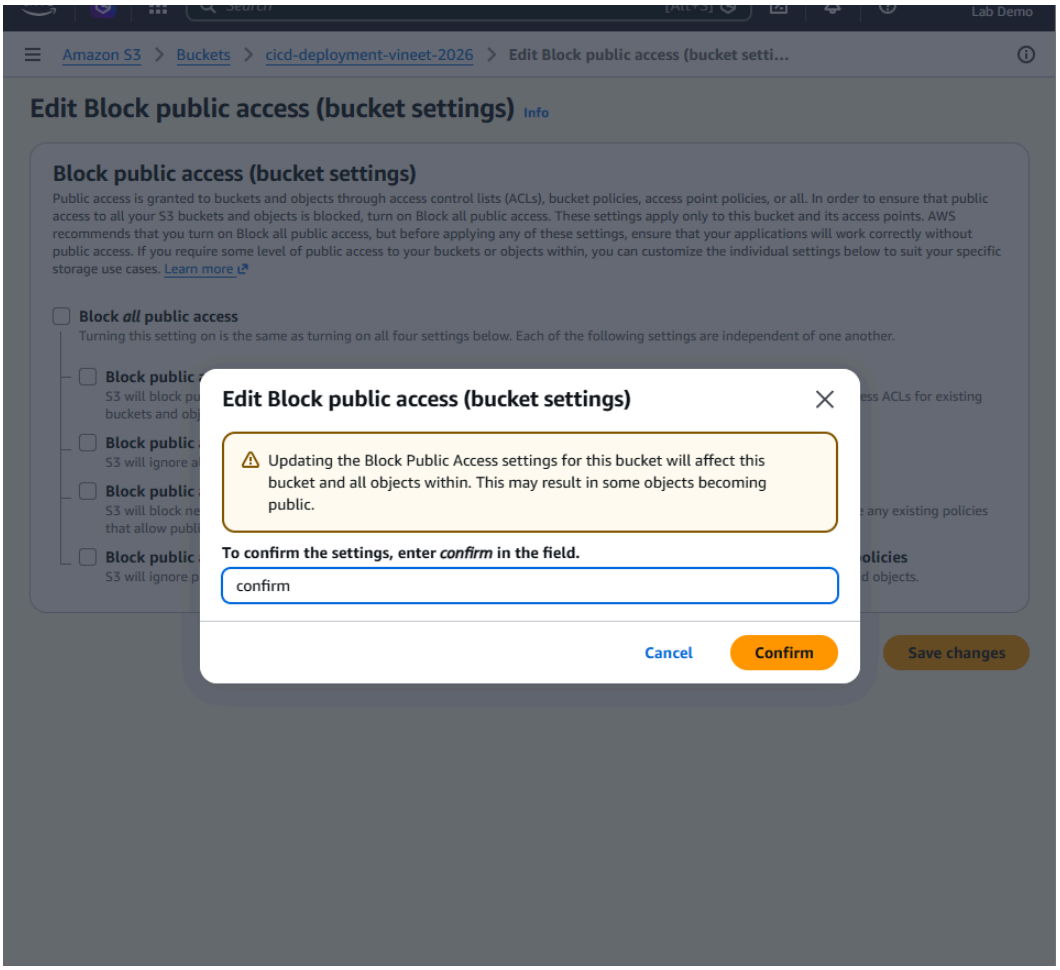
13. Enable Static Website Hosting:

In the properties of the deployment bucket cicd-deployment-vineet-2026, static website hosting is enabled so that S3 serves index.html as the default document through a public website endpoint.



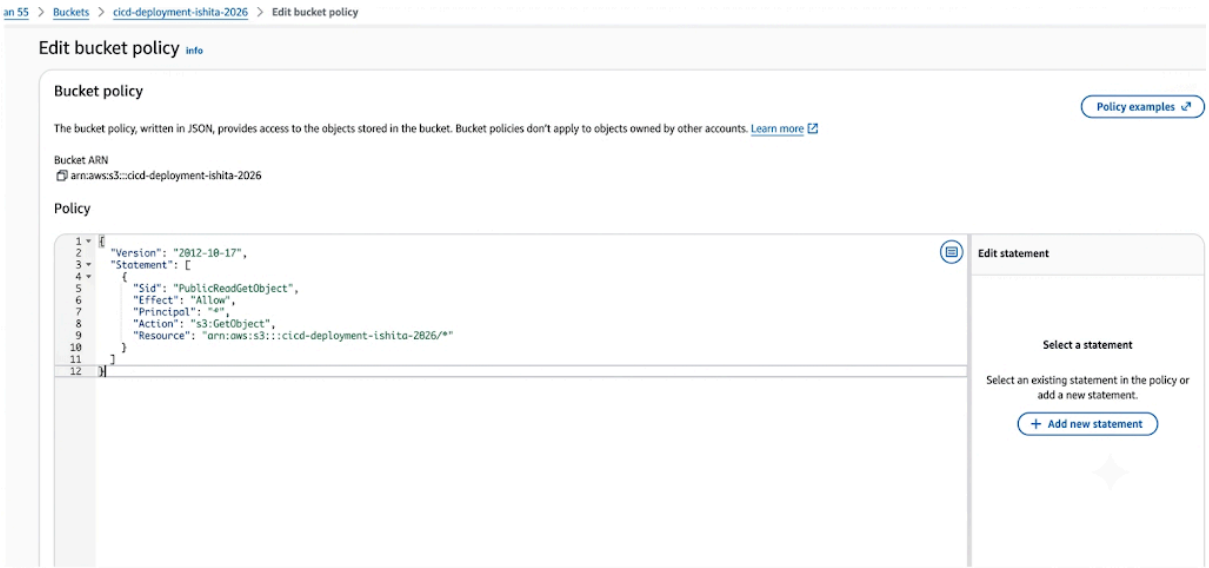
14. Disable Block Public Access:

S3 blocks all public access to new buckets by default. To let visitors load the website, Block all public access is turned off for the deployment bucket, and the change is acknowledged by typing confirm.



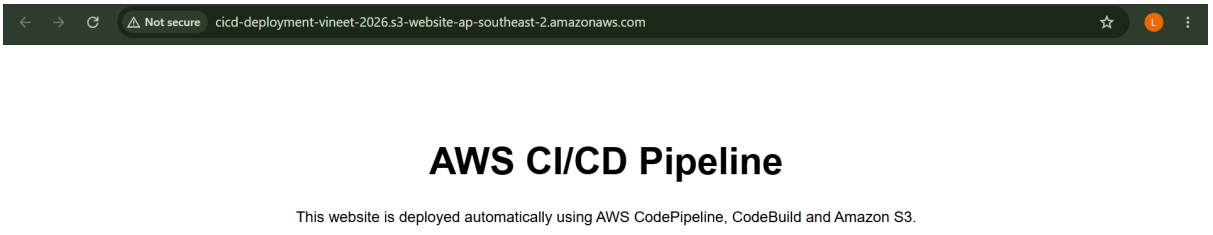
15. Attach a Bucket Policy:

A bucket policy is added that allows the s3:GetObject action for every principal on all objects in the bucket. This grants anonymous read-only access to the website files without allowing any uploads or deletions.



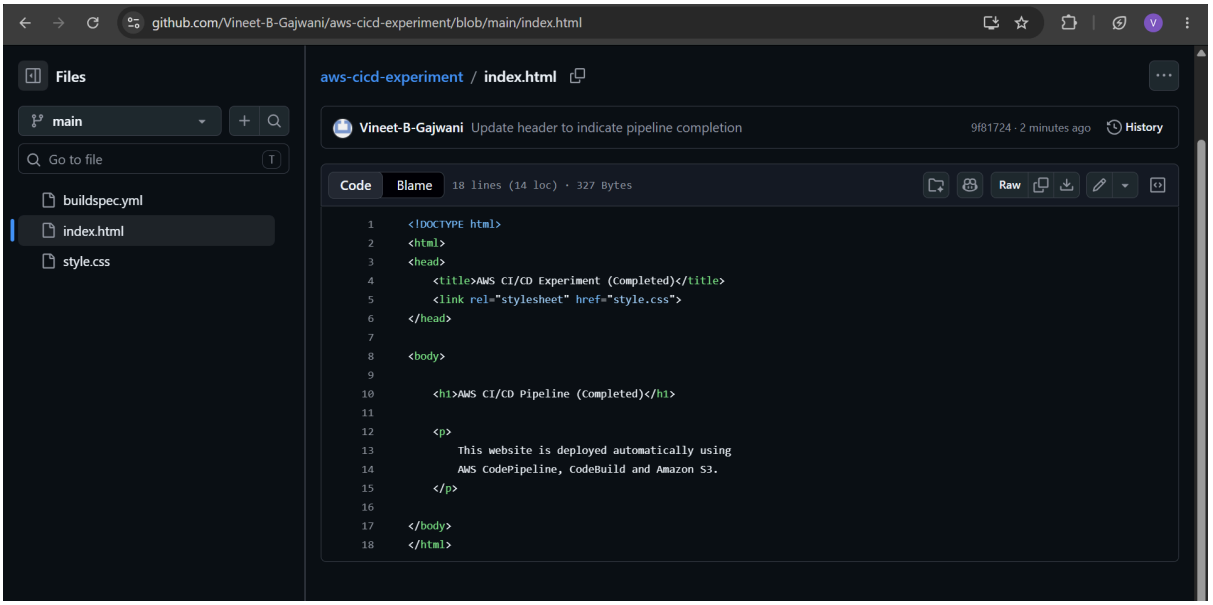

16. Access the Deployed Website:

Opening the S3 website endpoint in a browser displays the page titled AWS CI/CD Pipeline, confirming that the files built by CodeBuild were successfully deployed to S3 by the pipeline.



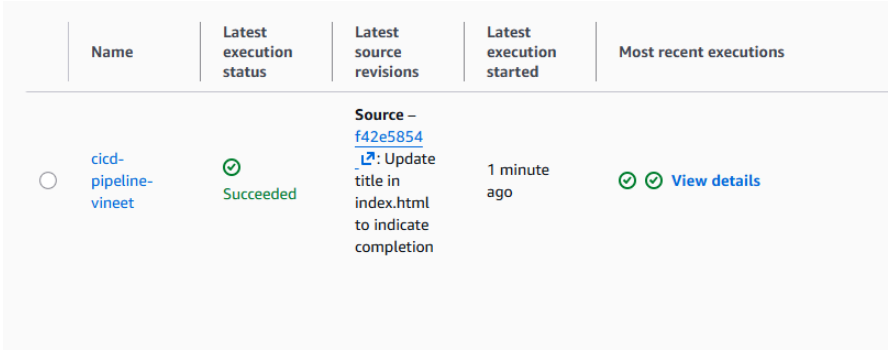
17. Push an Update to the Repository:

To test continuous deployment, index.html is edited on GitHub so that the heading reads AWS CI/CD Pipeline (Completed), and the change is committed directly to the main branch.



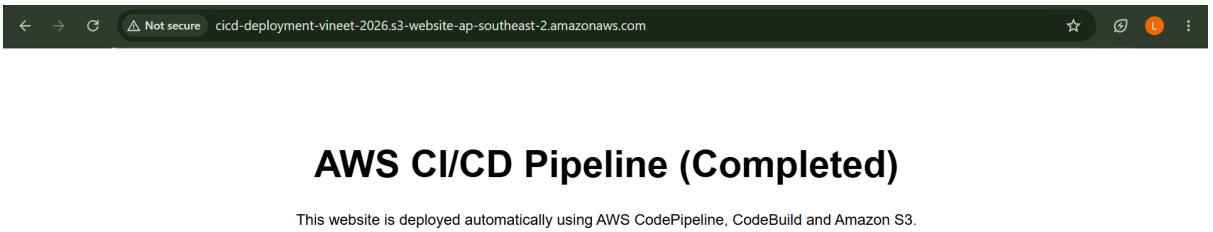

18. Observe the Automatic Pipeline Trigger:

Without any manual action, the pipeline detects the new commit and starts a fresh execution. The pipeline list shows the latest execution status as Succeeded along with the commit message of the update.



19. Verify the Website After the Update:

Refreshing the website endpoint now shows the updated heading AWS CI/CD Pipeline (Completed). The change travelled from a GitHub commit to the live site entirely through the automated pipeline.




## Conclusion:

In this experiment, a complete CI/CD pipeline was built on AWS using CodePipeline, CodeBuild, and Amazon S3. An artifact bucket was created first, followed by a CodeBuild project connected to a GitHub repository through OAuth and configured to read its build steps from buildspec.yml. A V2 pipeline was then assembled with three stages — a GitHub source stage on the main branch, a CodeBuild build stage, and an Amazon S3 deploy stage — and its first execution completed successfully, with the same commit flowing through every stage.

The deployment bucket was then turned into a public static website by enabling static website hosting, disabling Block Public Access, and attaching a read-only bucket policy, after which the deployed page was reachable through the S3 website endpoint. Finally, an edit to index.html committed on GitHub triggered the pipeline automatically and the updated heading appeared on the live site without any manual steps. Overall, the experiment demonstrated the core idea of continuous delivery: once the pipeline is defined, every commit is built and deployed automatically, making releases fast, consistent, and repeatable.